

CSE 590 Annotated Bibliography

Yi Wang

April 15, 2026

1 Introduction

Static analysis is a broad area in computer science concerned with reasoning about program behavior without executing the program. It plays a fundamental role in compiler optimization, bug detection, and security analysis. An example of program behavior that we can learn about is constant propagation: whether certain variables are constant across all executions. Precisely computing program behavior is often infeasible due to features such as branching, loops, and recursion, which introduce complex and potentially unbounded execution paths. As a result, practical analyses rely on sound approximations of program behavior.

Abstract interpretation provides a principled framework for constructing such approximations. It models program semantics using abstract domains that simplify the space of possible values while preserving correctness guarantees. Within this framework, dataflow analysis has emerged as one of the most widely used and effective techniques. Dataflow analysis represents programs as control-flow graphs (CFGs), where information is propagated across program points and combined at merge locations. This process can be formalized as computing fixpoints of monotone functions over structured domains, typically lattices, which ensure that the analysis converges and remains well-defined.

In this project, we examine the development of this field, starting from foundational works that establish the theoretical framework of dataflow analysis and abstract interpretation, and progressing to later works that focus on improving the efficiency and scalability of the underlying algorithms.

2 Important Results

The early foundations of static analysis are established by Kildall (1973) and Cousot and Cousot (1977). Kildall introduces a general framework for modeling programs as CFGs and

formulating dataflow problems as systems of equations over lattices, solved via iterative fixpoint computation. This work provides the first systematic way to compute precise program properties statically. Cousot and Cousot (1977) later generalized this approach through abstract interpretation, shifting the focus from exact values to sound approximations. This work was also fundamental by addressing the fundamental limitation of capturing precise values, which is often infeasible. By introducing abstract domains and formal abstraction mechanisms, they provide a unified framework which generalized Kildall’s framework to approximations over any abstract domains.

Subsequent work focuses on formalizing, improving, and extending this framework. Hecht and Ullman (1975) study the efficiency of fixpoint algorithms, showing how both evaluation order and lattice height impacts convergence and providing an improved algorithm, initiating the long line of work on boosting the efficiency of fixpoint algorithms.

Some fundamental improvements to the fixpoint algorithm were developed relatively early. Kam and Ullman (1977) mentioned a better way to order vertices, introduced an improved worklist algorithm over the naive round-robin algorithm. Their work did not formally state the algorithm, but the worklist algorithm is widely used, and future works include formal implementations of it. Cooper, Harvey and Kennedy (2002) implemented the worklist algorithm using different data structures to store the worklist, and compared the performances when running on actual programs. Another important optimization tool is given by Tarjan (1972), when he introduced a linear-time algorithm for computing SCC decompositions. Tarjan’s work doesn’t directly apply the SCC decompositions to fixpoint algorithms in CFGs, but many subsequent works utilize Tarjan’s algorithm as a subprotocol, providing more efficient fixpoint computations by isolating cyclic dependencies in CFGs.

3 Open Questions

Despite decades of progress, several challenges remain in the field of static analysis and fixpoint computation, focused on improving its scalability, precision, and applicability.

First, there is an inherent tradeoff between precision and efficiency. Intuitively, more precise analysis require more detailed abstract domains (take computing exact values as an extreme case), which can lead to high computational costs or even non-terminating programs. To better understand this tradeoff is still an active area of research.

Second, these techniques become much harder to apply to real programs. Features like function calls, concurrency, and dynamic behavior make the analysis significantly more complex. Although existing frameworks provide a foundation, it is still difficult to make analysis both precise and efficient.

Finally, people are still working on improving fixpoint algorithms, including better ordering strategies, incremental analysis, and parallelization. Whether it is possible to choose the

optimal ordering strategy is a very interesting question.

4 Project Implementation

I will implement the fixpoint algorithm and efficiency improvements on a toy programming language. The language will have the basic statements, branching, and looping, so that the control-flow of programs is still nontrivial to examine.

For the input to my implementation, I will either choose to parse the original program (harder), or restrict the input to an abstract syntax tree (easier). Either way, I will first construct a CFG from the input.

For the choice of the abstract domain, I will include an option for constant propagation and also sign analysis. I might also add a feature that allows choosing different levels of approximation: for example, I can add more precision to constant propagation by allowing conclusions such as $x \in \{5, 6\}$, instead of setting $x = \top$ once x can take more than one value.

To implement the fixpoint algorithm once the CFG is constructed and the abstract domain is fixed, I will first implement the naive round-robin algorithm, and then improvements using the worklist algorithm, improvements using SCC decomposition, and possibly both. For the worklist algorithm, I will implement the worklist using different data structures.

I will analyze the performance of my implementation by testing on different inputs, and compare the runtime of the different implementations.

References

- [1] K. D. Cooper, T. J. Harvey, and K. Kennedy, “Iterative dataflow analysis, revisited,” Rice University, Tech. Rep., 2002.

This paper revisits the problem of efficiently computing fixpoints for dataflow analysis in practice, in particular the efficiency of algorithms that use the worklist algorithm. The paper studies both the theory and practice of iterative dataflow analysis, explaining the role of reducibility in the classic Kam-Ullman bound and comparing different iterative solver variants experimentally. Its main contribution is to show that implementation choices, such as the data structure choice in the worklist design, can have a major effect on performance, even when the underlying complexity analysis is the same. A limitation is that the paper only focuses on improvements by changing the choice of data structures, which is a minor modification. Future work includes other implementation detail improvements, and comparing results using different evaluation techniques.

- [2] P. Cousot and R. Cousot, “Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints,” in *Proc. POPL’77*, 1977.

This paper addresses the problem of how to systematically design program analyses that go beyond reasoning about exact program values, which was the focus on earlier works. The authors introduce abstract interpretation as a general framework that enables the construction of analyses based on abstractions of program behavior. The key idea to making sound approximations is to map concrete program semantics to abstract domains that capture only relevant properties, such as signs or intervals. The majority of the paper gives a rigorous formalization of this concept. A limitation of this paper is that it doesn’t mention implementation or efficiency concerns. Future work includes developing concrete abstractions and various implementation techniques for scalability.

- [3] M. S. Hecht and J. D. Ullman, “A simple algorithm for global data flow analysis problems,” in *Proc. POPL’75*, 1975.

This paper focuses on the problem of improving efficiency of fixpoint algorithms designed for solving dataflow problems. While earlier work established the framework, the authors analyze specific iterative algorithms, particularly worklist-based methods, and study their convergence behavior. The solution involves analyzing how the order which the nodes are evaluated affect performance, and improving efficiency based on that. The evaluation is analytical, with formal complexity analysis and comparisons of algorithmic strategies. A limitation is that the analysis assumes relatively simple program structures, and does not fully address real life large-scale systems. Future work includes other techniques on improving efficiency of fixpoint algorithms, such as SCC decompositions, and developing more scalable implementations.

- [4] J. B. Kam and J. D. Ullman, “Monotone data flow analysis frameworks,” *Acta Informatica*, 1977.

This paper addresses the problem of establishing formal correctness and convergence guarantees for dataflow analysis frameworks. The authors introduce the monotone framework, specifying conditions under which iterative algorithms converge to a maximal fixed point. They distinguish between the meet-over-all-paths (MOP) solution and the maximal fixed point (MFP) solution, and characterize when these coincide. The author provides formal proofs for convergence, as well as other correctness properties. Importantly, they mention how the ordering of evaluation affects the efficiency, which would later be formalized as the worklist algorithm. A limitation is that the framework assumes monotonicity and finite-height lattices, which may restrict applicability. Future work includes extending these guarantees to more complex domains and improving precision when MFP differs from MOP.

- [5] G. A. Kildall, “A unified approach to global program optimization,” in *Proc. POPL’73*, 1973.

This paper is one of the first works in static analysis, studying the problem of global program optimization by computing precise dataflow information. The author defines a general framework which models programs as CFGs and formulates dataflow problems as systems of equations over lattice structures, and proposes an iterative algorithm that computes fixpoints of these equations to determine program properties. The author provides formal mathematical definitions and proofs, and illustrates applicability by giving multiple optimization examples. A limitation is that no specific implementation is provided (except for the small examples), and efficiency was not discussed. Future work includes developing more detailed algorithms and optimizing efficiency.

- [6] R. E. Tarjan, “Depth-first search and linear graph algorithms,” *SIAM Journal on Computing*, 1972.

This paper addresses the problem of efficiently analyzing the structure of directed graphs, in particular, identifying strongly connected components (SCCs). The author introduces depth-first search (DFS)-based algorithms that compute SCCs in linear time, and provides formal proofs of correctness and runtime. While this is a general graph algorithm not specific to program analysis, this solution is highly relevant for optimizing dataflow algorithms by isolating cycles in CFGs. The evaluation consists of algorithmic analysis and proofs of correctness and linear-time complexity. A limitation is that the work does not directly address program analysis applications. Future work (in the context of static analysis) involves integrating these graph techniques into dataflow frameworks to improve fixpoint computation efficiency.